

	MANUAL DE INSTRUCCIONES	I523-IT-1
	Guía de Desarrollo Seguro para Aplicaciones Internas de Prodismo SRL – Nivel Básico	REVISIÓN: 0 FECHA: 25/08/25
		PAG.: 1 / 8

Guía de Desarrollo Seguro para Aplicaciones Internas de Prodismo SRL – Nivel Básico, alineada con el control **A.8.31 de ISO 27001:2022** (Separación de entornos de desarrollo, pruebas y producción). Dado que actualmente solo existen dos entornos (desarrollo local y producción pública), **la guía refuerza la separación mínima exigible y establece prácticas de seguridad esenciales para tecnologías Node.js, Express, MongoDB, Docker y Google Cloud Storage.**

1. Objetivo y alcance

Objetivo:

Establecer un conjunto de prácticas mínimas y listas de verificación que permitan desarrollar aplicaciones ERP internas de forma segura, garantizando la separación básica de entornos y reduciendo riesgos de vulnerabilidades comunes.

Alcance:

- Aplicaciones tipo ERP desarrolladas internamente y expuestas a internet.
- Tecnologías: Node.js, Express, JavaScript, HTML, CSS, MongoDB (NoSQL), Docker, Google Cloud Storage para imágenes.
- Entornos: desarrollo (local) y producción (público).

2. Principios de seguridad básica

1. **Defensa en profundidad:** no confiar en una única capa de seguridad.
2. **Mínimo privilegio:** usuarios, procesos y contenedores deben ejecutarse con los permisos estrictamente necesarios.
3. **Seguridad por diseño:** incorporar controles desde la etapa de codificación, no al final.
4. **Separación de entornos:** desarrollo y producción deben estar aislados lógicamente y físicamente (red, credenciales, datos).
5. **Registro y monitoreo:** todos los eventos de seguridad deben ser trazables.

	MANUAL DE INSTRUCCIONES	I523-IT-1
	Guía de Desarrollo Seguro para Aplicaciones Internas de Prodismo SRL – Nivel Básico	REVISIÓN: 0 FECHA: 25/08/25
		PAG.: 2 / 8

3. Separación de entornos de desarrollo y producción (A.8.31)

Aunque no existe un entorno de pruebas formal, se exige una separación mínima para evitar impactos no deseados.

Área	Desarrollo (local)	Producción
Infraestructura	Máquina local, contenedores Docker con redes internas	Servidores en la nube, acceso restringido por usuario/password
Credenciales	Variables de entorno con datos de prueba (nunca reales)	Secrets manager (variables de entorno cifradas, etc.)
Base de datos	MongoDB local o contenedor; datos ficticios o anonimizados	MongoDB en producción con autenticación fuerte
Almacenamiento de imágenes	Bucket de prueba en Google Cloud Storage (con políticas restringidas)	Bucket de producción con políticas de acceso controladas y logs habilitados
Control de cambios	Rama develop o feature/* en repositorio	Rama main o master con despliegue automatizado tras revisión manual
Acceso	Libre para desarrolladores	Solo personal autorizado

Checklist – Separación de entornos:

- No se utilizan credenciales de producción en código fuente o en desarrollo.
- Los contenedores de desarrollo no exponen puertos innecesarios.
- Los buckets de Google Cloud Storage de desarrollo y producción son diferentes y sus claves de acceso están separadas.

	MANUAL DE INSTRUCCIONES	I523-IT-1
	Guía de Desarrollo Seguro para Aplicaciones Internas de Prodismo SRL – Nivel Básico	REVISIÓN: 0 FECHA: 25/08/25
		PAG.: 3 / 8

- Se utiliza un sistema de control de versiones con ramas protegidas (ej. main con reglas de aprobación).
- No hay acceso directo desde el entorno de desarrollo a la base de datos de producción.

4. Buenas prácticas de codificación

4.1 Node.js / Express

Seguridad en rutas y middlewares:

- Usar helmet para configurar cabeceras HTTP seguras.
- Limitar el tamaño de las peticiones con `express.json( { limit: '1mb' } );`
y `express.urlencoded( { extended: true, limit: '1mb' } );`
- Implementar rate limiting (ej. `express-rate-limit`) en rutas críticas y públicas.
- Validar todos los inputs en el servidor (no confiar en validaciones del lado del cliente).

Checklist Node.js/Express:

- helmet está instalado y configurado.
- Se utiliza `express-rate-limit` con valores adecuados (ej. 100 solicitudes por 15 minutos por IP).
- No se exponen stack traces en producción (`NODE_ENV=production` y `app.use` de errores personalizado).
- Las cookies sensibles tienen `httpOnly`, `Secure` y `SameSite=Strict` o `Lax`.

4.2 MongoDB (NoSQL)

Prevención de inyecciones NoSQL:

- Usar el driver nativo con operadores seguros (ej. `new ObjectId()` cuando corresponda).
- Evitar concatenar directamente variables en consultas; emplear `filter` con parámetros tipados.
- Validar que los datos recibidos cumplan con el esquema esperado (puede usarse `mongoose` si aplica).

	<u>MANUAL DE INSTRUCCIONES</u>	I523-IT-1
	Guía de Desarrollo Seguro para Aplicaciones Internas de Prodismo SRL – Nivel Básico	REVISIÓN: 0 FECHA: 25/08/25
		PAG.: 4 / 8

Controles adicionales:

- Crear índices únicos para campos que deben ser únicos (evitar duplicados).
- Usar roles de base de datos con privilegios mínimos para la aplicación.

Checklist MongoDB:

- Se valida y escapa toda entrada que forme parte de consultas.
- La aplicación se conecta con un usuario dedicado con permisos solo sobre las colecciones necesarias.
- Las operaciones de actualización usan proyecciones para modificar solo campos permitidos.
- No se exponen mensajes de error detallados de la base de datos al cliente.

4.3 JavaScript, HTML, CSS

- **XSS (Cross-Site Scripting):**
 - Escapar contenido dinámico antes de insertarlo en el DOM (usar **textContent** en lugar de **innerHTML**).
 - Aplicar CSP (Content Security Policy) mediante helmet para limitar fuentes de scripts y estilos.
- **Almacenamiento en cliente:**
 - No almacenar tokens sensibles en localStorage; **usar cookies httpOnly** en su lugar.
- **CSS:**
 - Evitar usar url() con datos no confiables.

Checklist frontend:

- No se utiliza innerHTML con datos no sanitizados.
- La política CSP está configurada (ej. default-src 'self').
- No hay información sensible en el código fuente visible en el navegador.
- Las sesiones se manejan con tokens en cookies seguras.

	<u>MANUAL DE INSTRUCCIONES</u>	I523-IT-1
	Guía de Desarrollo Seguro para Aplicaciones Internas de Prodismo SRL – Nivel Básico	REVISIÓN: 0 FECHA: 25/08/25
		PAG.: 5 / 8

5. Seguridad en almacenamiento de imágenes (Google Cloud Storage)

- **Buckets separados:** usar un bucket para desarrollo y otro para producción.
- **Políticas de acceso:**
 - Las imágenes subidas por usuarios deben validarse (tipo MIME, tamaño máximo, escaneo antivirus opcional).
 - Los nombres de objetos deben ser aleatorios o hashados para evitar enumeración.
 - Para imágenes públicas, se deben firmar URLs con tiempo de expiración si no deben ser accesibles indefinidamente.
- **Autenticación:** usar cuentas de servicio con permisos limitados (solo escritura/lectura según necesidad).

Checklist Google Cloud Storage:

- El bucket de producción no permite listar objetos públicamente.
- Se valida el tipo de archivo en el servidor (ej. usando file-type o verificando cabeceras).
- Las claves de cuenta de servicio se rotan periódicamente y nunca se incluyen en el código.
- Las URLs firmadas se usan para acceso temporal si las imágenes no son públicas.
- Se registran las operaciones de escritura/eliminación en logs de auditoría.

6. Gestión de dependencias y contenedores (Docker)

- **Dependencias:**
 - Escanear periódicamente con npm audit y snyk o herramientas similares.
 - Mantener actualizadas las librerías, priorizando parches de seguridad.
 - No usar dependencias obsoletas o con vulnerabilidades conocidas.
- **Docker:**
 - Usar imágenes base oficiales y ligeras (ej. node:18-slim).
 - Ejecutar contenedores con usuario no root.

	MANUAL DE INSTRUCCIONES	I523-IT-1
	Guía de Desarrollo Seguro para Aplicaciones Internas de Prodismo SRL – Nivel Básico	REVISIÓN: 0 FECHA: 25/08/25
		PAG.: 6 / 8

- No exponer puertos internos innecesarios.
- Construir imágenes con --no-cache para evitar capas antiguas con secretos.
- Escanear imágenes con herramientas como docker scan o Trivy antes de desplegar.

Checklist dependencias y Docker:

- Se ejecuta npm audit al menos una vez por mes y se resuelven vulnerabilidades de nivel alto/crítico.
- El Dockerfile no copia node_modules como directorio externo; se usa COPY y RUN npm ci con --only=production.
- El contenedor se ejecuta con un usuario no root (crear usuario app).
- Las variables de entorno sensibles nunca están hardcodeadas en la imagen; se inyectan en tiempo de ejecución.
- Las imágenes se escanean antes de subir al registro.

7. Revisiones de código y análisis estático

Se introduce un proceso mínimo:

- **Revisión manual (code review):**
 - Toda modificación que se fusione a la rama **main** debe ser revisada por al menos otro desarrollador.
 - Se debe verificar especialmente: validación de entradas, manejo de errores, exposición de secretos, lógica de autenticación/permisos.
- **Análisis estático automatizado:**
 - Incorporar un linter (ESLint) con reglas de seguridad (eslint-plugin-security).
 - Si el repositorio está en GitHub, activar Dependabot para alertas de vulnerabilidades.

Checklist revisiones:

- Existe un repositorio con rama main protegida (requiere aprobación).
- Cada pull request contiene una descripción de los cambios y verificación de checklist de seguridad.

	MANUAL DE INSTRUCCIONES	I523-IT-1
	Guía de Desarrollo Seguro para Aplicaciones Internas de Prodismo SRL – Nivel Básico	REVISIÓN: 0 FECHA: 25/08/25
		PAG.: 7 / 8

- Se utiliza ESLint con reglas de seguridad.

8. Listas de verificación consolidadas

Checklist general antes del despliegue a producción

- Se ha verificado que no hay credenciales de producción en el código o en variables de entorno del contenedor.
- Las variables de entorno se almacenan de forma segura (secrets manager) y no en archivos planos dentro del repositorio.
- La aplicación se ejecuta con NODE_ENV=production y los mensajes de error están deshabilitados.
- Se han revisado las dependencias con npm audit y no hay vulnerabilidades críticas.
- El contenedor se ejecuta con usuario no root y se ha escaneado con herramienta de seguridad.
- La base de datos de producción está aislada y utiliza autenticación fuerte.
- El bucket de Google Cloud Storage tiene políticas de acceso adecuadas y se validan los archivos subidos.
- Se ha realizado una revisión de código manual de los cambios incluidos.
- Existe un plan de reversión en caso de incidente.

Checklist de desarrollo diario

- Se trabaja en una rama separada desde develop o main (según flujo).
- Antes de subir código, se ejecuta npm audit y se corrigen vulnerabilidades.
- Se han añadido validaciones para todas las entradas de usuario.
- Las consultas a MongoDB utilizan el driver correctamente sin concatenar strings.
- Las cookies de sesión están marcadas como httpOnly, Secure y SameSite.
- No se ha dejado código comentado ni console.log en producción.
- Se ha probado manualmente que los errores no exponen información interna.

	MANUAL DE INSTRUCCIONES	I523-IT-1
	Guía de Desarrollo Seguro para Aplicaciones Internas de Prodismo SRL – Nivel Básico	REVISIÓN: 0 FECHA: 25/08/25
		PAG.: 8 / 8

9. Recomendaciones para futuras mejoras

Aunque el nivel de seguridad actual es básico, se sugiere:

1. **Implementar un entorno de pruebas (staging)** que refleje producción, para validar cambios de forma aislada.
2. **Automatizar escaneo de seguridad** en el pipeline CI/CD (SAST, análisis de composición de software).
3. **Establecer una política de rotación de secretos** (claves API, cuentas de servicio).
4. **Realizar capacitaciones periódicas** en desarrollo seguro.
5. **Configurar logs centralizados y alertas** para detección temprana de anomalías en producción.

Aprobación y vigencia

Esta guía entra en vigencia a partir de su comunicación. Cualquier desviación debe ser justificada y aprobada por el responsable de seguridad o líder técnico.
